Chapter 7 worked solutions

GPT, BERT, and T5

This guide covers the written solutions for Exercises 1–5 and 12. The companion Jupyter notebook works through Exercises 6–11 and is also available as a PDF. We use the chapter's teaching convention of one token per word in the examples below.

1. Move the answer one place ahead

Start with *the red car stopped*. When we have read only *the*, the next word in this example is *red*. Once we have read *the red*, the next word is *car*. GPT makes both predictions in one training pass by pairing each input position with the word that follows it in the original text.

Array	Tokens in order
Input	[the, red, car]
Next-token labels	[red, car, stopped]

To build the input, keep everything except the last token. To build the labels, keep everything except the first token. Both arrays have length three. In the chapter's GPT example, the first word supplies the starting context and the last word supplies the last label. This construction uses the words alone.

An output can read input positions at or before its own position. Positions start at zero in the following table.

Output position	Input at that position	Readable input, in order	Label
0	the	the	red
1	red	the red	car
2	car	the red car	stopped

At output position 1, the diagonal attention cell lets the query read input position 1, which contains *red*. Its label is *car*. The occurrence of *car* in the input is at position 2, and the causal mask blocks that position from query 1. The label array supplies the correct answer to the loss after the model computes its scores. It contributes no keys or values to attention.

The restriction is applied at every layer. For example, a representation at position 0 cannot first read position 2 and then pass that information to position 1 in a later layer. Position 0 itself has access only to position 0. In general, with zero-based numbering, output position i reads input positions 0 through i and predicts the original token at position $i + 1$. The diagonal gives access to the current input, which is already part of the prefix.

Now take a new five-token sentence, *the old car moved today*. Its input and label arrays each have four entries.

Array	Tokens in order
Input	[the, old, car, moved]
Next-token labels	[old, car, moved, today]

Output position	Input at that position	Readable input, in order	Label
0	the	the	old
1	old	the old	car
2	car	the old car	moved
3	moved	the old car moved	today

The same rule pairs *moved* with *today* in the last row. In this example *today* appears only in the label array. In general, N words supply $N - 1$ pairs under this shift.

2. Draw the information paths

Read each grid one row at a time. The **query** is the position being updated. The **keys** are the positions it can compare against and whose associated value vectors it can read. A check mark permits that read. A dash blocks it. The marks specify the allowed positions before the attention weights are computed.

Causal self-attention. Queries and keys come from the same sequence. Query 0 can read key 0, query 1 can read keys 0 and 1, and the pattern continues. For a query at position i and a key at position j, the rule is $j \le i$.

Query / key	0	1	2	3	Permitted
0	✓	—	—	—	1
1	✓	✓	—	—	2
2	✓	✓	✓	—	3
3	✓	✓	✓	✓	4

The diagonal and the cells below it give $1 + 2 + 3 + 4 = 10$ permitted reads. For instance, the blocked cell in row 1 and column 2 prevents the query at position 1 from reading the later position 2.

Bidirectional self-attention. With all four positions valid, every query can read all four keys, including keys later in the sequence.

Query / key	0	1	2	3	Permitted
0	✓	✓	✓	✓	4
1	✓	✓	✓	✓	4
2	✓	✓	✓	✓	4
3	✓	✓	✓	✓	4

The count is $4 \times 4 = 16$. The six additional cells are exactly the ones above the causal diagonal.

Cross-attention. Let D_i denote a decoder query and E_j an encoder key. There are three rows for decoder positions and four columns for encoder positions. Each decoder position can read all valid encoder positions.

Decoder query / encoder key	E_0	E_1	E_2	E_3	Permitted
D_0	✓	✓	✓	✓	4
D_1	✓	✓	✓	✓	4
D_2	✓	✓	✓	✓	4

The shape is 3×4, with 12 permitted cells. The rectangle is possible because decoder and encoder positions belong to different sequences. For example, decoder query D_0 can read the last encoder key E_3. The causal rule for decoder self-attention still limits which *decoder* positions D_0 can read.

Suppose that E_3 is padding and the first three encoder positions are real tokens. The tensor can retain four columns for batching, while the padding mask blocks the entire fourth column for this example.

Decoder query / encoder key	E_0	E_1	E_2	E_3 (pad)	Permitted
D_0	✓	✓	✓	—	3
D_1	✓	✓	✓	—	3
D_2	✓	✓	✓	—	3

All three decoder queries lose the same key, leaving $12 - 3 = 9$ permitted cells. When computing each attention row, the padded key receives zero weight and the weights sum to one over the remaining valid keys.

3. Separate selection from replacement

Think of two decisions for each eligible token. First, decide whether its position will be scored. For each selected position, decide what the model will see there. In the chapter's rule, the first decision selects 15% in expectation. The second uses 80% mask, 10% random, and 10% unchanged *among the selected positions*.

For 200 eligible tokens, the expected number selected is $200 \times 0.15 = 30$. To compute the expected count for a particular treatment, multiply its conditional probability by 30. For example, the probability that an eligible position is selected **and** receives the mask treatment is $0.15 \times 0.80 = 0.12$. Among 200 eligible positions that gives 24 mask replacements in expectation.

Treatment at a selected position	Probability given selection	Expected count
Supply [MASK]	80%	$30 \times 0.80 = 24$
Draw a random replacement token	10%	$30 \times 0.10 = 3$
Supply the original token unchanged	10%	$30 \times 0.10 = 3$
All selected positions	100%	30

The expected number of scored labels is **30**. Each selected position contributes one label, and that label is always the original token at the same position. In the teaching helper, unselected positions carry the special loss value **IGNORE** (−100). Those positions can still provide context through attention.

Suppose the original word at a selected position is *red*. With the mask treatment the model reads [MASK] there, and the loss scores *red*. With the random treatment the model might read *blue*, and the loss still scores *red*. With the unchanged treatment it reads *red* and the loss scores *red* because the position was selected in the first decision.

The random branch can also draw the original ID. If its draw is *red*, the input looks the same as in the unchanged branch even though the sampled treatments differ. The teaching helper draws uniformly from ten ordinary words, so conditional on entering that branch, it draws the original word with probability 1/10. The expected count of three random treatments includes those draws. Counting visible changes would miss both the unchanged branch and random draws equal to the original.

Expected counts describe an average over repeated samples. For example, with 30 selected positions a run might produce 22 mask, four random, and four unchanged treatments. The three counts still add to 30. A selection procedure that samples each eligible position at the stated rate may also vary the total selected. The chapter's helper receives the selected positions explicitly and samples only the treatments, so its selected count is fixed by the caller.

4. Put the fragments back in order

Start with *the red car stopped near the old bridge today*. Number every word from zero. A half-open span (a, b) covers positions from a up to, and excluding, b.

Position	Word	Action
0	the	Keep in source
1	red	Remove in span $(1, 3)$
2	car	Remove in span $(1, 3)$
3	stopped	Keep in source
4	near	Keep in source
5	the	Keep in source
6	old	Remove in span $(6, 8)$
7	bridge	Remove in span $(6, 8)$
8	today	Keep in source

The two removed spans are *red car* and *old bridge*. Replace each whole span in the encoder input by one **sentinel**, a dedicated vocabulary token that identifies the gap. S_0 identifies the first gap and S_1 identifies the second. Append **EOS** to mark the end of the source.

To form the target, emit S_0 followed by *red car*, then S_1 followed by *old bridge*. Append the extra sentinel S_2 to close the final removed fragment under the chapter's convention, then append **EOS** to end the target. The source contains the two gap sentinels. The closing sentinel belongs to the reconstruction target.

Sequence	Tokens in order	Count
Corrupted source	[the, S_0, stopped, near, the, S_1, today, EOS]	8
Reconstruction target	[S_0, red, car, S_1, old, bridge, S_2, EOS]	8
Shifted decoder input	[BOS, S_0, red, car, S_1, old, bridge, S_2]	8

The counts come from three different constructions. The source contains five intact words, two gap sentinels, and one end marker. The target contains four recovered words, three sentinels, and one end marker. For the decoder input, prepend **BOS** to the target and drop the target's last token from the input copy. That removes one entry and adds one, preserving length eight. The original target still ends with **EOS** and all eight labels are scored.

To see the shift at work, look at the aligned decoder rows.

Position	Decoder input	Target at this position
0	BOS	S_0
1	S_0	red
2	red	car
3	car	S_1
4	S_1	old
5	old	bridge
6	bridge	S_2
7	S_2	EOS

At position 1, the decoder has read **BOS** and S_0 and learns to predict *red*. It can also use cross-attention to read the encoded source, where S_0 identifies the gap between *the* and *stopped*. At position 7 it has read through S_2 and learns to predict **EOS**. This separation lets the decoder see the source context on both sides of a gap while keeping future reconstruction labels outside its causal prefix.

The words *stopped near the* stay visible in the corrupted source. They can help the encoder represent the two surrounding gaps. The reconstruction target asks for the missing fragments and their boundary markers, so those already supplied words are left out of the target. In the teaching vocabulary, the sentinel names appear as **S0**, **S1**, and **S2**, and the decoder has a dedicated **BOS** token.

5. Check a loss before training

A **logit** is a score for one vocabulary token before softmax converts the scores to probabilities. Consider a single output row. If each of the V logits is zero, then each numerator in softmax is $\exp(0) = 1$. The denominator is the sum of V ones. Every token, including the reference label y, receives the same probability.

$$p(y) = \frac{1}{V}, \qquad -\log p(y) = -\log\left(\frac{1}{V}\right) = \log V.$$

Each evaluated row contributes $\log V$ to the negative log-likelihood, using the natural logarithm. In the teaching vocabulary, $V = 17$, so the contribution is $\log 17 \approx 2.833213$. With N evaluated rows, the loss sum is $N \log 17$. The mean divides by the same N. The table uses the full value of $\log 17$ before rounding each result.

Objective	Evaluated labels N	Loss sum $N \log 17$	Mean loss
GPT next-token example	3	8.499640	2.833213
BERT masked-token example	1	2.833213	2.833213
T5 span reconstruction	8	22.665707	2.833213

In GPT, there is one evaluated label for each shifted pair. In BERT, the model can produce four output rows in the sentence example, and the loss reads only the selected row. In T5, the four missing words, three sentinels, and **EOS** all contribute. The sum differs across the three objectives because their numbers of evaluated labels differ.

Appending two ignored padding labels increases the stored length of each target by two. For GPT, for instance, the target can now hold [red, car, stopped, IGNORE, IGNORE]. The loss reads three entries and divides by three. The BERT and T5 examples still use one and eight evaluated entries, respectively. Therefore the reported sums and means remain exactly as before. The pad rows contribute neither a term in the numerator nor a position in the denominator. Dividing GPT's sum by five would incorrectly give about 1.699928, even if its three real rows were scored correctly.

PAD is an input token ID used to fill out a batch. **IGNORE** tells the loss to skip a particular label row. If a padding row were instead labeled with the vocabulary ID **PAD** and evaluated, its uniform prediction would add another $\log 17$ to the sum. The mean could still equal $\log 17$ in this uniform test, since the mistakenly scored row also has the same loss. Checking the evaluated count and the loss sum along with the mean makes that error visible.

Equal mean losses here follow from the artificial uniform scores. To assess what a trained model has learned, we would need scores on held-out data under the relevant prediction objective. GPT predicts continuations, BERT recovers selected tokens from corrupted inputs, and T5 reconstructs missing spans. The arithmetic check verifies that those objectives count and reduce their label rows as intended.

12. Design a fair comparison (optional)

Suppose the input is *"The red car moved near the bridge, and the blue van stopped by the gate."* The task is to identify the color of the object in the clause with *moved*. The answer is **red**. Now swap the colors: *"The blue car moved near the bridge, and the red van stopped by the gate."* The answer becomes **blue**, although the sentence still contains the same words. To answer correctly in both cases, the model must connect the color to the relevant verb.

Build each scene from two distinct objects and locations, with one object assigned to *moved* and the other to *stopped*. Write the same scene four ways: use both color assignments and both orders of the two clauses. Each scene therefore has two red and two blue answers, and each color is the correct answer once in each clause position. Vary the objects, places, and connective templates across scenes, always keeping the same rule for producing the label. These four variants form a **scene group**. They stay together even when an equivalent wording or clause order is possible. This blocks shortcuts such as picking the first color or learning that a particular vehicle always has a red answer.

Create 2,500 distinct scene groups and split by group before generating the four variants. Use 2,000 groups (8,000 sentences) for training, 250 (1,000 sentences) for validation, and 250 (1,000 sentences) for the final test. Key the split on the underlying objects, locations, and their roles; exclude duplicate or paraphrased scenes across splits. This measures generalization to new scene combinations from the same controlled task. It does not by itself measure transfer to natural writing. Exclude these task sentences and their variants from pretraining as well.

Use one tokenizer and the same sentence text for all three models. Set a length limit that fits every sentence and its boundary markers. Padding positions must not become valid keys or pooled positions. For each family, choose one hidden vector to represent the sentence, then learn a linear classification head that maps the vector to two logits:

Family	Vector used by the head	Information available to that vector
GPT	Causal hidden vector at an end marker appended after the sentence	All sentence tokens are earlier than the marker.
BERT	Bidirectional hidden vector at a classification marker placed before the sentence	The marker can attend to all valid sentence tokens.
T5	First decoder hidden vector from a fixed start token	Cross-attention reads the encoder's representation of the whole sentence.

In the T5 version, the decoder input contains just the fixed start token. Keep this position valid even in implementations that reuse the padding ID as the decoder start token. Supplying the answer as part of the decoder input would change the classification problem. In all versions use logit 0 for red and logit 1 for blue. Train with two-class cross-entropy on the scene labels and predict the class with the larger logit. The same two possible answers and the same scoring rule apply to every model.

For a comparison of pretrained families, train the three backbones with their own objectives on the same underlying text corpus and tokenizer, using a common compute budget such as 10^{16} training FLOPs per backbone. FLOPs count floating-point operations; estimate them consistently for the forward and backward passes, including both T5 stacks. Aim for 10 million trainable parameters per model, within 5% including its task head, and report the actual parameter counts. This design uses one pretraining checkpoint per family, so the adaptation runs estimate variation from adaptation seeds. Equal FLOPs and equal parameter counts do not imply equal numbers of tokens seen or labels scored, so record those counts too. The budget is an example of a rule to set before the study, not a report of a run.

During adaptation, initialize a new task head and fine-tune the whole model in every family. Use the same optimizer type, the same set of two candidate learning rates (for example, 10^{-4} and 3×10^{-4}), the same three run seeds, and a budget of 10^{14} training FLOPs per adaptation run. Check validation accuracy at predetermined fractions of that budget, giving every candidate the same number of checkpoint evaluations. Fix the selection rule in advance: choose the learning rate and checkpoint fraction with the highest mean validation accuracy across seeds, break a tie by choosing the earlier fraction, and then choose the lower learning rate if needed.

Seal the test split until the selection rule has been applied and all settings are fixed. Evaluate each of the three selected checkpoints per family on that split once. The primary metric is sentence accuracy, the number of correct color predictions divided by 1,000. Report the mean and spread across seeds. For an uncertainty interval on a difference between two families, resample entire test scene groups and score both families on the same sampled groups; the four variants from one scene are related observations. This interval describes variation from sampling test scenes for these fitted models; the spread across seeds describes variation between adaptation runs. A useful secondary metric is the fraction of scenes with all four variants correct. If the test reveals a weakness that motivates another round of tuning, use validation data to develop the change and reserve a new sealed test set for the new comparison.

Three features of raw pretraining losses prevent them from settling this question:

1. **The models are graded on different events.** GPT scores next tokens, BERT scores selected tokens, and T5 scores a sequence of removed tokens plus reconstruction markers. The mix of targets, and which positions enter the mean, differ. A sum also grows with the number of graded labels.
2. **The predictions use different evidence.** GPT sees a prefix; BERT can see words on both sides, and some selected inputs retain the answer; T5 reads the corrupted source and earlier target tokens. Even a prediction of the same word can pose a different question under those conditions.
3. **The task measures a new outcome.** A smaller pretraining loss measures better prediction under that objective and evaluation setup. It does not tell us how often the adapted model links the correct color to *moved* in new scenes. That requires scoring the same held-out classification examples with the same metric.

A comparison under this protocol could support a claim about the measured task, split, and budget. It would not establish that one family is best for every downstream task.